

TECHNICAL REFERENCE

Unitree L1

File Organization Reference

Ownership, provenance, purpose, and interactions
A complete guide to the maintained source tree

First edition
Prepared 29 July 2026
Validated project baseline 07616c2
Audience: Developers, maintainers, reviewers, operators, and system integrators

DOCUMENT CONTROL

This document is generated from `file-organization-reference.md`. Commands and paths describe the repository state identified above.

Operate the powered LiDAR only after reading the safety section.

About this reference

This document explains how the Unitree L1 ROS 2 and OctoMap project is organized at project commit 07616c2. It identifies every file tracked by that commit, distinguishes project-owned source from pinned upstream source and generated data, and records the purpose and consumer of each maintained artifact.

The reference is intentionally more detailed than the repository overview. It is meant to answer practical maintenance questions: where a change belongs, which file installs another file, which layer owns a parameter, how an ignored dependency is reconstructed, why a generated directory must not enter source control, and which components interact at build time and at run time.

All paths are relative to the repository root unless explicitly shown as container paths. A path ending in / denotes a directory. Git stores files, not empty directories, so each directory in the tree exists because it contains at least one file or because a reconstruction or runtime command creates it.

ENGINEERING NOTE

The authoritative snapshot for the 73-file inventory is commit 07616c2; the later manual sources, publication tools, and three tracked PDFs are outside that snapshot.

Snapshot facts

Property	Verified value
Project commit	07616c2833f4b838c0686c9f1a50d9fdc87f6583
Short commit	07616c2
Tracked files	73
Tracked bytes	124,469
Project ROS 2 packages	l1_bringup, l1_monitor, l1_octomap_bringup
Reconstructed ROS 2 packages	octomap_mapping, octomap_server, unitree_lidar_ros2
Unitree pin	1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6
OctoMap pin	f79da9a9a1fcdf82e72dab4df288d6cc27c6e163
Required ROS environment	ROS 2 Humble on Ubuntu 22.04 inside Docker
Host role	Docker client, file owner, serial-device provider, and X11 server

What the ownership terms mean

Term	Meaning	Update authority	Git policy
Project-owned	Authored and maintained in this repository	Project maintainers	Tracked
Pinned upstream	Authored by Unitree or OctoMap and imported at an immutable commit	Upstream project; local tree must stay unchanged	Ignored by outer Git
Container-supplied	Installed from the pinned base image or Jammy/Humble package repositories	Image recipe and upstream package maintainers	Recipe tracked; installed files untracked
Generated build data	Produced by colcon, CMake, setuptools, compilers, or tests	Tooling	Ignored
Runtime data	Produced by a sensor run, recorder, evaluator, or map saver	Operator and runtime tools	Ignored
Publication output	Produced from maintained manual source	Manual renderer and reviewers	Three named PDFs tracked
Local state	Credentials, editor files, caches, X11 cookies, or machine-specific settings	Local user and tools	Ignored

Project-owned does not mean every dependency is reimplemented. The project deliberately owns the integration boundary: container policy, parameter values, launch composition, monitoring, validation, recording, mapping workflow, and documentation. Unitree owns the driver and SDK. OctoMap owns the occupancy server and saver. ROS and Ubuntu own the binary platform supplied inside the image.

Three provenance classes

At the source-control boundary, the detailed terms reduce to three principal classes: project-owned files tracked by the outer repository, pinned upstream files reconstructed as ignored nested repositories, and generated or local files excluded from maintained source. Container-supplied software is reconstructed from the project-owned image recipe and belongs to the execution environment rather than the source tree.

Structural principles

The layout follows a small set of rules that should remain stable as the project grows.

1. The outer Git repository contains authored source, reproducible configuration, tests, and concise documentation.
2. Third-party repositories are reconstructed from immutable URLs and commits rather than copied into project history.
3. Vendor code is not patched in place. Adaptations are made in a project-owned package or wrapper.
4. ROS execution is supported only in the project container.
5. GUI and hardware access are opt-in Compose overlays, not properties of the base service.
6. Colcon outputs belong at the `ros2_ws/` root and never beneath `ros2_ws/src/`.
7. Bags, maps, test logs, caches, and non-manual exports remain untracked products; the three reviewed manual PDFs are tracked publications.
8. Host-facing scripts provide the supported operational interface and enforce preconditions before invoking ROS.
9. Safety, provenance, and validation claims remain readable without requiring access to old machine-specific logs.

SAFETY / WARNING

A directory named `build`, `install`, or `log` anywhere below `ros2_ws/src` violates the project structure even though `.gitignore` would hide it.

Exact project tree at commit 07616c2

The following tree names every tracked file and shows the two ignored upstream locations and the standard ignored output locations in context.

```
Unitree-L1-ROS2-Octomap/
|-- .dockerignore
|-- .gitignore
|-- LICENSE
|-- README.md
|-- config/
|   '-- dependencies.repos
-- docker/
|   |-- Dockerfile
|   |-- compose.gui.yaml
|   |-- compose.lidar.yaml
|   |-- compose.yaml
|   '-- entrypoint.sh
-- docs/
|   |-- decisions.md
|   |-- hardware-runbook.md
|   |-- project-structure.md
|   |-- sources.md
|   |-- validation-matrix.md
|   '-- versions-lock.md
-- ros2_ws/
|   |-- colcon_defaults.yaml
|   |-- build/                                [ignored, generated]
|   |-- install/                             [ignored, generated]
|   |-- log/                                 [ignored, generated]
|   '-- src/
|       |-- ll_bringup/
|       |   |-- config/
|       |   |   |-- unitree_ll.rviz
|       |   |   '-- unitree_ll.yaml
|       |   |-- ll_bringup/
|       |   |   '-- __init__.py
|       |   |-- launch/
|       |   |   '-- unitree_ll.launch.py
|       |   |-- resource/
|       |   |   '-- ll_bringup
|       |   |-- test/
|       |   |   |-- test_flake8.py
|       |   |   |-- test_launch_file.py
|       |   |   '-- test_pep257.py
|       |   |-- package.xml
|       |   |-- setup.cfg
|       |   '-- setup.py
|       |-- ll_monitor/
|       |   |-- ll_monitor/
```

```

|-- __init__.py
|-- monitor_node.py
'-- stats.py
-- resource/
'-- ll_monitor
-- test/
|-- test_flake8.py
|-- test_pep257.py
'-- test_stats.py
-- package.xml
-- setup.cfg
'-- setup.py
-- ll_octomap_bringup/
|-- config/
|   |-- ll_octomap.rviz
|   |-- octomap.yaml
|   '-- saved_octomap.rviz
|-- ll_octomap_bringup/
|   |-- __init__.py
|   -- launch/
|       |-- ll_octomap.launch.py
|       |-- unitree_ll_octomap.launch.py
|       '-- view_saved_octomap.launch.py
|   -- resource/
|       '-- ll_octomap_bringup
|   -- test/
|       |-- test_flake8.py
|       |-- test_launch_file.py
|       |-- test_package_metadata.py
|       '-- test_pep257.py
|   -- package.xml
|   -- setup.cfg
|   '-- setup.py
-- octomap_mapping/                                [ignored, pinned upstream clone]
'-- unilidar_sdk/                                    [ignored, pinned upstream clone]
-- scripts/
|-- assert-ros-container.sh
|-- bag-info.sh
|-- check-lidar.sh
|-- docker-build.sh
|-- docker-shell.sh
|-- evaluate-octomap.sh
|-- fetch-dependencies.sh
|-- gui-smoke-test.sh
|-- inspect-octomap.sh
|-- lidar-launch.sh
|-- lidar-validate.sh
|-- monitor-synthetic-test.sh
|-- octomap-launch.sh
|-- record-bag.sh
|-- replay-bag.sh
|-- save-octomap.sh
|-- smoke-test.sh
|-- verify-docker-only.sh
|-- view-octomap.sh
'-- workspace-build.sh
-- bags/                                            [ignored, runtime]
-- exports/                                        [ignored at baseline; reviewed PDFs tracked later]
-- logs/                                           [ignored, runtime validation]
'-- maps/                                          [ignored, runtime]

```

The tree contains four tracked files at the root, one in `config/`, five in `docker/`, six in `docs/`, thirty-seven in `ros2_ws/`, and twenty executable scripts. The thirty-seven workspace files consist of `colcon_defaults.yaml` plus eleven files in `ll_bringup`, ten in `ll_monitor`, and fifteen in `ll_octomap_bringup`.

Root control files

Root inventory

Path	Origin and owner	Purpose	Installed or consumed by
<code>.dockerignore</code>	Project-owned	Defines a minimal and deterministic Docker build context	Docker build engine
<code>.gitignore</code>	Project-owned	Defines generated, reconstructed, runtime, cache, credential, and editor exclusions	Git and repository hygiene checks
<code>LICENSE</code>	Project-owned	MIT licence for project-authored software and documentation	Contributors, redistributors, and reviewers
<code>README.md</code>	Project-owned	Concise entry point for status, setup, build, hardware, RViz, bag, and map workflows	Operators and maintainers

`.gitignore`

The first policy block ignores `build/`, `install/`, and `log/` at every depth. This broad form is deliberate: an accidental `colcon` command from `ros2_ws/src` must not cause generated files to appear as candidate source. `compile_commands.json` is also excluded because it is a generated compiler index.

The next block ignores only the two dependency destinations beneath `ros2_ws/src`. Their exact revisions remain visible in `config/dependencies.repos`, while their full trees remain independent nested Git repositories. Outer Git therefore records the recipe for retrieving them, not duplicate copies of their content.

Runtime exclusions cover `bags/`, non-manual `exports/`, `logs/`, and `maps/` at repository root, plus files ending in `.log` or `.zip`. Narrow exceptions admit only the three named PDFs beneath `exports/manuals/`. Python bytecode, package metadata, coverage output, `pytest` and type-checker caches, local environment files, X11 authorization files, IDE state, operating-system metadata, and backup files are also excluded.

An ignore rule does not make a path valid. In particular, a hidden `ros2_ws/src/build/` remains structurally wrong and causes `workspace-build.sh` to fail. Ignore rules protect source control; wrapper checks protect the physical layout.

`.dockerignore`

The Docker context begins by excluding `**`, then re-admits only the `docker/` directory, `docker/Dockerfile`, and `docker/entrypoint.sh`. The image recipe copies only the entrypoint. Project source, local dependencies, bags, maps, credentials, build products, and repository history are never sent to the image builder.

At run time, Compose bind-mounts the repository at `/workspace`. This separates image construction from source iteration: rebuilding project packages does not require rebuilding the base image, and building the base image does not capture local runtime data.

`LICENSE`

The MIT text applies to project-owned files. The ignored Unitree and OctoMap clones carry their own upstream licence files and notices. Their licensing terms must be evaluated from those pinned trees when redistributing a complete assembled environment.

`README.md`

The README is intentionally operational rather than exhaustive. It records the supported Docker-only architecture, current dependency pins, the six-package workspace, generated paths, build commands, live LiDAR sequence, RViz sequence, bag workflow, OctoMap workflow, and links to the focused documents under `docs/`.

The README should remain the shortest reliable route to a successful build. Detailed rationale belongs in `docs/decisions.md`; safety and hardware detail belongs in `docs/hardware-runbook.md`; exhaustive file provenance belongs in this reference.

Dependency lock

`config/dependencies.repos`

Unitree property	Locked value
Repository key	<code>unilidar_sdk</code>
Upstream URL	<code>https://github.com/unitreerobotics/unilidar_sdk.git</code>
Pinned commit	<code>1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6</code>
Destination	<code>ros2_ws/src/unilidar_sdk/</code>

OctoMap property	Locked value
Repository key	octomap_mapping
Upstream URL	https://github.com/OctoMap/octomap_mapping.git
Pinned commit	f79da9a9a1fcdf82e72dab4df288d6cc27c6e163
Destination	ros2_ws/src/octomap_mapping/

The file uses `vcstool` repository syntax. `scripts/fetch-dependencies.sh` feeds it to `vcs import --skip-existing` when either destination is absent. The script then independently checks both nested Git HEADs and rejects any local modification, deletion, or untracked content reported by the nested repository.

The manifest is project-owned configuration even though the referenced content is upstream-owned. Changing a commit is a dependency upgrade and requires a build, test, compatibility review, version-lock update, and relevant validation-matrix update.

ENGINEERING NOTE

Never replace a commit pin with a moving branch or tag if exact reconstruction is required.

Container definition

The `docker/` directory is the boundary between the Ubuntu 24.04 host and the supported ROS 2 environment. The host provides Docker, project files, the selected serial character device, X11 authorization, and DRI devices. ROS packages and ROS executables remain inside Ubuntu 22.04.

Docker file inventory

Path	Origin and meaningful contents	Installed or consumed by
<code>docker/Dockerfile</code>	Project-owned recipe: digest-pinned Humble/Jammy base, apt dependencies, non-root user, and entrypoint	Docker build engine
<code>docker/compose.yaml</code>	Project-owned base service, bind mount, ROS domain, colcon defaults, and security restrictions	All Compose-based scripts
<code>docker/compose.gui.yaml</code>	Project-owned X11 cookie/socket, DRI devices, and video/render groups	GUI shell, RViz, GUI smoke test
<code>docker/compose.lidar.yaml</code>	Project-owned mapping for one selected tty and supplementary tty group	Live L1 runtime
<code>docker/entrypoint.sh</code>	Project-owned executable that sources ROS and the workspace overlay before executing the command	Every container process

`docker/Dockerfile`

The default base reference is `ros:humble-ros-base-jammy` at digest

`sha256:5c793b92e0b12d6babb438cb20eed7766495fde6419a21e3d2e918464f09dc17`. A digest prevents the base-image tag from silently resolving to a different image.

The apt layer adds the compiler and CMake toolchain, PCL and Boost development packages, OctoMap libraries, ROS 2 launch and TF components, point-cloud conversions, OctoMap messages and ROS bridge, rosbag2, ros2doctor, RViz2, colcon, rosdep, vcstool, Git, USB and network diagnostics, and X11/DRI utilities.

Build arguments carry the host UID and GID into the image. The recipe creates user `ros`, owns `/workspace`, switches away from root, selects `/workspace/ros2_ws` as the working directory, and updates rosdep as that user. The image entrypoint is copied to `/usr/local/bin/unitree-entrypoint`.

The image does not contain the project source or either source dependency. Those arrive through the run-time bind mount. It also does not pin every apt package to a repository snapshot, so dependency source and base image are reproducible while a future apt layer is not guaranteed to be byte-for-byte identical.

[docker/compose.yaml](#)

The base service is named `dev` under Compose project `unitree-l1`. It builds or uses image `unitree-l1:humble-v1.0.16`, initializes an init process, supports interactive terminals, and works at `/workspace/ros2_ws`.

`COLCON_DEFAULTS_FILE` points to `/workspace/ros2_ws/colcon_defaults.yaml`. `ROS_DOMAIN_ID` defaults to 42 but may be overridden. The entire repository root is bind-mounted to `/workspace`, which makes generated files owned by the host-matched non-root user.

The base service drops all Linux capabilities and sets `no-new-privileges:true`. It deliberately contains no display mount, GPU mapping, serial mapping, privileged mode, or host networking. This lets builds and headless tests run without hardware or a graphical session.

[docker/compose.gui.yaml](#)

This overlay requires `DISPLAY` and a readable host `XAUTHORITY` file. It maps `/tmp/.X11-unix` and the authorization cookie read-only. It maps `/dev/dri` and supplies the actual host group IDs for the primary video and render devices.

The design avoids `xhost +` and avoids copying a mutable authorization file into the image. The overlay must be present when a container is created; `docker exec` cannot retroactively add the X11 mounts or DRI devices.

SAFETY / WARNING

`OCTOMAP_RVIZ=true` works only when the existing runtime was created with the GUI overlay, which the supported wrapper does through `START_RVIZ=true ./scripts/lidar-launch.sh`.

[docker/compose.lidar.yaml](#)

The hardware overlay maps the resolved host `LIDAR_DEVICE` to the stable in-container name `/dev/unitree_lidar`. It adds the device's actual group ID rather than broadening permissions. The container receives

`LIDAR_PORT=/dev/unitree_lidar`.

The host-side launcher verifies that the resolved object is a character device whose basename matches `tttyUSB*` or `tttyACM*`. The overlay itself never assumes that the same host tty number will survive a reconnect.

[docker/entrypoint.sh](#)

The entrypoint always sources `/opt/ros/${ROS_DISTRO}/setup.bash`. If `/workspace/ros2_ws/install/setup.bash` exists, it also sources the project overlay. Finally, `exec "$@"` replaces the shell with the requested command so signals reach the intended process.

This automatic overlay activation means a container created after a successful build can locate project and reconstructed packages without each Compose command repeating the source commands. Scripts that use `docker exec` still source explicitly where clarity or shell behavior requires it.

Maintained technical documentation

Documentation inventory

Path	Origin and owner	Purpose	Primary consumer
<code>docs/decisions.md</code>	Project-owned	Architecture decision record with rationale and limitations	Maintainers and reviewers
<code>docs/hardware-runbook.md</code>	Project-owned	Safety, connection, validation, recording, and replay	Hardware operators
<code>docs/project-structure.md</code>	Project-owned	Concise ownership and output policy	Contributors
<code>docs/sources.md</code>	Project-owned bibliography	Authoritative external technical references	Reviewers and future research
<code>docs/validation-matrix.md</code>	Project-owned	Reproducible checks and hardware-dependent status	Testers and maintainers
<code>docs/versions-lock.md</code>	Project-owned	Immutable inputs and validated compatibility	Builders and release maintainers

[docs/decisions.md](#)

The decision record explains why the project uses Jammy/Humble Docker on a Noble host, keeps vendor code unchanged, separates hardware and GUI overlays, avoids host networking by default, mounts X11 authorization read-only, and adds project-owned bringup and monitor layers.

It also records the diagnostic QoS and provisional thresholds, ROS domain isolation, rosdep exceptions, package discovery restriction, OctoMap integration, stationary versus mobile mapping, the boundary between occupancy mapping and pose estimation, and the controlled map-saving policy.

New decisions belong here when they change an architectural constraint or assign ownership between layers. Routine commands and troubleshooting should not be added as decisions.

[docs/hardware-runbook.md](#)

The runbook starts with the physical hazards and power/interface facts needed before connecting the L1. It describes stable serial discovery, the separate power supply, the 2,000,000-baud driver expectation, least-privilege group access, the first headless run, data validation, RViz, recording, replay, shutdown, and reconnection.

The runbook distinguishes node existence from data success because the pinned vendor driver can stay alive even if initialization did not yield data. `LIDAR_DATA_VALIDATION_PASS`, not a visible node name alone, is the supported live-data verdict.

Hardware procedure changes belong here when they affect safe setup, device identification, or live-operation sequence. Machine-specific device IDs and raw validation logs do not belong here.

[docs/project-structure.md](#)

This concise document states the same central ownership boundary as this reference without the file-by-file detail. It lists tracked responsibilities, reconstructed dependencies, generated paths, the required out-of-source layout, clean build commands, the six-package inventory, and source-tree hygiene checks.

It is suitable for a contributor who needs the rules quickly. This reference is suitable for an auditor or maintainer who needs to understand every artifact and interaction.

[docs/sources.md](#)

The source register identifies official ROS 2 Humble platform and QoS documentation, official Docker image and Compose references, Unitree SDK and L1 manual sources, OctoMap release and server references, colcon configuration and discovery documentation, and candidate Point-LIO sources.

The file records which L1 manual pages support power, adapter, UART, scan, and IMU facts. It separates documented nominal rates from measured project rates. Copied PDFs are intentionally absent; the maintained source contains links and relevance notes instead.

[docs/validation-matrix.md](#)

The matrix assigns identifiers to configuration, dependency, discovery, output-isolation, build, test, smoke, monitor, Docker-boundary, GUI, hardware, recording, OctoMap, map-lifecycle, and future mobile-mapping checks.

`PASS` means a reproducible check succeeded. `PASS_HW` means the result required the physical sensor. A pending mobile-mapping row remains explicit because the occupancy server is not a pose estimator.

The bottom of the file records static commands that should find no forbidden tracked artifacts or source-tree outputs. When an operational claim changes, update its check and status rather than adding unverifiable prose elsewhere.

[docs/versions-lock.md](#)

The lock records the base-image name and digest, Ubuntu and ROS versions, Unitree and OctoMap releases and commits, and the validated compiler, PCL, Eigen, Boost, OctoMap, Python, CMake, RViz2, colcon, rosdep, and vcstool environment.

It also documents the reproducibility boundary: base image and Git dependencies are immutable, while individual apt packages are not repository-snapshot pinned. Dependency upgrades must update the lock and the vcstool manifest together.

Manual publication sources and tooling

The present documentation set adds nine project-owned files after the audited `07616c2` runtime baseline: six maintainable source/tool files and three reviewed PDF publications. They do not change ROS behavior. The editable sources and renderer prevent the tracked PDFs from becoming opaque artifacts.

Path	Origin and owner	What it contains	Producer or consumer
docs/manuals/README.md	Project-owned	Manual inventory, render command, output policy, and tool requirements	Documentation maintainers
docs/manuals/engineering-manual.md	Project-owned	Full architecture, operation, validation, troubleshooting, and maintenance reference	PDF renderer and engineers
docs/manuals/first-run-tutorial.md	Project-owned	Guided UniLiDAR, RViz2, stationary OctoMap, save, and reopen workflow	PDF renderer and first-time operators
docs/manuals/file-organization-reference.md	Project-owned	This ownership, provenance, purpose, and interaction reference	PDF renderer and maintainers
scripts/render-manual.py	Project-owned	Constrained Markdown parser and shared A4 groff visual system	scripts/build-manuals.sh
scripts/build-manuals.sh	Project-owned executable	Atomic rendering, A4 bounds, text-marker, font, and checksum validation	Documentation maintainers
exports/manuals/UNITREE_L1_ENGINEERING_MANUAL.pdf	Project-authored publication	Reviewed engineering-manual PDF	Engineers and GitHub readers
exports/manuals/UNITREE_L1_FIRST_RUN_TUTORIAL.pdf	Project-authored publication	Reviewed first-run PDF	Operators and GitHub readers
exports/manuals/UNITREE_L1_FILE_ORGANIZATION_REFERENCE.pdf	Project-authored publication	Reviewed file-reference PDF	Maintainers and GitHub readers

The renderer accepts only the documented Markdown subset used by these three sources. It creates an intermediate groff and PostScript representation in a temporary directory. The build wrapper then publishes:

```
exports/manuals/UNITREE_L1_ENGINEERING_MANUAL.pdf
exports/manuals/UNITREE_L1_FIRST_RUN_TUTORIAL.pdf
exports/manuals/UNITREE_L1_FILE_ORGANIZATION_REFERENCE.pdf
```

The editable Markdown and two generic rendering tools are maintained source. The three exact PDF paths are explicit .gitignore exceptions and are tracked for direct GitHub access; every other export remains ignored. Rebuild the complete set from the repository root with `./scripts/build-manuals.sh`.

ENGINEERING NOTE

With six source/tool files and three PDFs committed after 07616c2, the tracked inventory becomes 82 files. The runtime baseline remains the 73-file commit documented in the reconciliation section.

ROS 2 workspace control

`ros2_ws/colcon_defaults.yaml`

This is the only tracked file directly beneath `ros2_ws/`. It controls package discovery and canonical output locations inside the mandatory container.

Colcon verb	Controlled input or output
Global	log-base: /workspace/ros2_ws/log
list	Five permitted base paths yielding six packages
build	Absolute build and install roots, symlink install, cohesive console output
test	Same discovery roots and absolute build, install, and result paths
test-result	/workspace/ros2_ws/build

The five discovery roots are the three project packages, the OctoMap repository root, and only the Unitree ROS 2 driver subdirectory. The OctoMap root contains two valid ROS 2 packages, which is why five base paths yield six packages.

The Unitree repository also contains a ROS 1 catkin package and a raw CMake SDK. Pointing colcon at the entire Unitree repository would expose components that do not belong in this ROS 2 build. The narrow Unitree base path is therefore a structural control, not a performance optimization.

The file uses absolute container paths so its output rules remain correct even when a user invokes colcon from `/workspace/ros2_ws/src`. Compose exports its location through `COLCON_DEFAULTS_FILE`.

EXPECTED RESULT

colcon list reports exactly `l1_bringup`, `l1_monitor`, `l1_octomap_bringup`, `octomap_mapping`, `octomap_server`, and `unitree_lidar_ros2`.

Common anatomy of project Python packages

All three `l1_*` packages use `ament_python`. Their repeated packaging files have standard but necessary roles.

Package manifests

Path	Origin and owner	Purpose
<code>ros2_ws/src/l1_bringup/package.xml</code>	Project-owned	Manifest for driver bringup, monitor, and RViz integration
<code>ros2_ws/src/l1_monitor/package.xml</code>	Project-owned	Manifest for the diagnostics node
<code>ros2_ws/src/l1_octomap_bringup/package.xml</code>	Project-owned	Manifest for OctoMap launch, TF, and RViz integration

Each manifest supplies package name, version `0.1.0`, description, maintainer, MIT licence, repository URL, `ament_python` build type, runtime dependencies, and lint/test dependencies. Rosdep, colcon, the ament index, package managers, and reviewers consume these declarations.

The OctoMap bringup manifest intentionally depends on `l1_bringup` but does not directly depend on `unitree_lidar_ros2`. Its standalone launch consumes any compatible PointCloud2 stream. The combined launch gains the driver indirectly through `l1_bringup`.

Setuptools definitions

Path	Origin and owner	Purpose
<code>ros2_ws/src/l1_bringup/setup.py</code>	Project-owned	Installs marker, manifest, launch file, YAML, and RViz profile
<code>ros2_ws/src/l1_monitor/setup.py</code>	Project-owned	Installs Python package and <code>l1_monitor</code> console entry point
<code>ros2_ws/src/l1_octomap_bringup/setup.py</code>	Project-owned	Installs marker, manifest, three launch files, YAML, and two RViz profiles

The setup definitions are the authoritative mapping from source-tree files to the install overlay. A launch or configuration file that is added to source but omitted from `data_files` may work when referenced directly yet disappear from the installed package share. Tests and smoke checks should therefore exercise installed paths.

Script-install configuration

Path	Origin and owner	Purpose
<code>ros2_ws/src/l1_bringup/setup.cfg</code>	Project-owned	Sets develop and install script directories for <code>l1_bringup</code>
<code>ros2_ws/src/l1_monitor/setup.cfg</code>	Project-owned	Sets develop and install script directories for <code>l1_monitor</code>
<code>ros2_ws/src/l1_octomap_bringup/setup.cfg</code>	Project-owned	Sets develop and install script directories for <code>l1_octomap_bringup</code>

These standard ament Python settings place executable scripts beneath the package-specific `lib` directory expected by `ros2 run`. Only `l1_monitor` currently registers a console executable, but keeping consistent package scaffolding makes installed behavior predictable.

Ament resource markers

Path	Origin and owner	Purpose
<code>ros2_ws/src/l1_bringup/resource/l1_bringup</code>	Project-owned empty marker	Registers <code>l1_bringup</code> in the ament resource index
<code>ros2_ws/src/l1_monitor/resource/l1_monitor</code>	Project-owned empty marker	Registers <code>l1_monitor</code> in the ament resource index
<code>ros2_ws/src/l1_octomap_bringup/resource/l1_octomap_bringup</code>	Project-owned empty marker	Registers <code>l1_octomap_bringup</code> in the ament resource index

The files are empty by design. Their filenames carry the identity. They are installed by the corresponding `setup.py`; they are neither cache files nor deletion candidates.

Python package markers

Path	Origin and owner	Meaningful contents
ros2_ws/src/l1_ bringup/l1_bringup/____ init__.py	Project-owned	Describes the Unitree L1 launch package
ros2_ws/src/l1_ monitor/l1_monitor/____ init__.py	Project-owned	Describes the stream-monitor package
ros2_ws/src/l1_ octomap_bringup/l1_ octomap_bringup/____ init__.py	Project-owned	Describes the project OctoMap launch package

These files make their directories Python packages. Their short module docstrings also satisfy the documentation-style checks. They are maintained source and must not be confused with generated `__pycache__` directories.

Repeated style tests

Path	Origin and owner	Test function
ros2_ws/src/l1_ bringup/test/test_ flake8.py	Project-owned	Runs ament flake8 and fails on Python style errors
ros2_ws/src/l1_ monitor/test/test_ flake8.py	Project-owned	Runs ament flake8 and fails on Python style errors
ros2_ws/src/l1_ octomap_bringup/test/ test_flake8.py	Project-owned	Runs ament flake8 and fails on Python style errors
ros2_ws/src/l1_ bringup/test/test_ pep257.py	Project-owned	Runs ament PEP 257 docstring checks
ros2_ws/src/l1_ monitor/test/test_ pep257.py	Project-owned	Runs ament PEP 257 docstring checks
ros2_ws/src/l1_ octomap_bringup/test/ test_pep257.py	Project-owned	Runs ament PEP 257 docstring checks

Colcon discovers these pytest files through the ament Python test integration. The tests belong with the package they protect. Their output belongs under `ros2_ws/build/` and `ros2_ws/log/`, not beside the test source.

l1_bringup package

`l1_bringup` is the project-owned adapter around the pinned Unitree ROS 2 node. It does not copy or modify driver source. It supplies stable device naming, project parameter defaults, optional monitoring, optional raw visualization, and a Docker-only launch guard.

Complete package inventory

Path	Owner	Purpose	Installed or consumed by
ros2_ws/src/l1_bringup/package.xml	Project	Manifest and dependency contract	rosdep, colcon, ament
ros2_ws/src/l1_bringup/setup.py	Project	Package and data-file install definition	setuptools and colcon
ros2_ws/src/l1_bringup/setup.cfg	Project	Script install paths	setuptools
ros2_ws/src/l1_bringup/resource/l1_bringup	Project	Ament package marker	ament index
ros2_ws/src/l1_bringup/l1_bringup/__init__.py	Project	Python package marker	Python and setuptools
ros2_ws/src/l1_bringup/launch/unitree_l1.launch.py	Project	Driver, monitor, and raw-RViz composition	lidar-launch.sh, ROS launch
ros2_ws/src/l1_bringup/config/unitree_l1.yaml	Project	Driver and monitor parameters	Launch nodes
ros2_ws/src/l1_bringup/config/unitree_l1.rviz	Project	Raw-cloud RViz display profile	RViz2
ros2_ws/src/l1_bringup/test/test_launch_file.py	Project	Launch-interface and guard checks	pytest through colcon
ros2_ws/src/l1_bringup/test/test_flake8.py	Project	Python style check	pytest through colcon
ros2_ws/src/l1_bringup/test/test_pep257.py	Project	Docstring style check	pytest through colcon

launch/unitree_l1.launch.py

The launch file begins by requiring `.dockerenv`. A direct host launch raises a clear error and points users to `scripts/lidar-launch.sh`. This complements the host-side runtime assertion rather than relying on documentation alone.

The public launch arguments are `port`, `cloud_topic`, `imu_topic`, `cloud_frame`, `imu_frame`, `rviz`, and `monitor`. The port defaults from `LIDAR_PORT`, falling back to `/dev/ttyUSB0` for a direct container invocation. The supported host wrapper supplies `/dev/unitree_lidar`.

The launch starts `unitree_lidar_ros2_node` from the upstream `unitree_lidar_ros2` package. It applies the installed YAML file, then overlays public launch arguments so callers can change device, topic, or frame without editing source.

The optional `l1_monitor` node receives the same cloud and IMU topic arguments and the shared YAML. The optional RViz node loads the installed raw profile. The default is monitor enabled and RViz disabled, which supports a low-variable first hardware run.

config/unitree_l1.yaml

The driver section establishes `/dev/unitree_lidar`, yaw bias zero, range scale 0.001, range bias zero, range limits zero to 50 metres, cloud frame `unilidar_lidar`, cloud topic `unilidar/cloud`, 18 scans per cloud, IMU frame `unilidar_imu`, and IMU topic `unilidar/imu`.

The monitor section establishes absolute topic names, `/diagnostics`, a two-second report period, three-second message timeout, one-second allowed header age, five-hertz cloud warning threshold, twenty-hertz IMU warning threshold, and a one-hundred-message statistics window.

These diagnostic rates are conservative project alarm thresholds, not claimed sensor specifications. The upstream manual and measured rates are recorded separately in `docs/sources.md` and `docs/validation-matrix.md`.

config/unitree_l1.rviz

The raw visualization uses fixed frame `unilidar_lidar` because the upstream driver publishes no pose transform. It shows a grid, axes, and `/unilidar/cloud` as three-pixel points with Z-axis coloring and one-second decay.

The cloud subscription uses Reliable reliability, Volatile durability, Keep Last history, and depth ten. The camera is an orbit view centered on the sensor frame. This profile is for confirming raw geometry; it is not the map-frame OctoMap view.

test/test_launch_file.py

The focused package test reads the launch source and requires the serial-port argument, `LIDAR_PORT` environment fallback, monitor launch configuration, monitor package, Docker marker check, and Docker-only explanation. It protects the public operational contract from accidental simplification.

l1_monitor package

`l1_monitor` is a read-only observer. It does not filter, transform, republish, or modify sensor data. It publishes diagnostic summaries so a visible process cannot be mistaken for a healthy stream.

Complete package inventory

Path	Owner	Purpose	Installed or consumed by
<code>ros2_ws/src/l1_monitor/package.xml</code>	Project	Diagnostics dependencies and package metadata	<code>rosdep</code> , <code>colcon</code> , <code>ament</code>
<code>ros2_ws/src/l1_monitor/setup.py</code>	Project	Installs package and <code>l1_monitor</code> executable	<code>setuptools</code> and <code>colcon</code>
<code>ros2_ws/src/l1_monitor/setup.cfg</code>	Project	Script install paths	<code>setuptools</code>
<code>ros2_ws/src/l1_monitor/resource/l1_monitor</code>	Project	Ament package marker	<code>ament</code> index
<code>ros2_ws/src/l1_monitor/l1_monitor/__init__.py</code>	Project	Python package marker	Python and <code>setuptools</code>
<code>ros2_ws/src/l1_monitor/l1_monitor/stats.py</code>	Project	Dependency-free bounded stream statistics	<code>monitor_node.py</code> , unit tests
<code>ros2_ws/src/l1_monitor/l1_monitor/monitor_node.py</code>	Project	ROS subscriptions and diagnostic publication	<code>ros2 run</code> , <code>bringup</code> launch
<code>ros2_ws/src/l1_monitor/test/test_stats.py</code>	Project	Statistics behavior tests	<code>pytest</code> through <code>colcon</code>
<code>ros2_ws/src/l1_monitor/test/test_flake8.py</code>	Project	Python style check	<code>pytest</code> through <code>colcon</code>
<code>ros2_ws/src/l1_monitor/test/test_pep257.py</code>	Project	Docstring style check	<code>pytest</code> through <code>colcon</code>

l1_monitor/stats.py

`StreamStats` keeps arrival timestamps in a bounded deque. It separately records total received count, last ROS header stamp, last frame, optional point count, field names, non-increasing timestamps, and zero timestamps.

`frequency_hz()` calculates frequency from the first and last arrival in the current window. `arrival_age_sec()` uses monotonic host time so system-clock changes do not create false message-timeout values. `stamp_age_sec()` compares the latest non-zero ROS header stamp with the ROS clock.

The class has no ROS dependency. This keeps its behavior easy to test and makes it clear which calculations are generic and which behavior belongs to the ROS node.

`l1_monitor/monitor_node.py`

The node declares topics, output topic, timing, rate thresholds, and window size as parameters. It creates Reliable, Volatile, Keep Last subscriptions at depth ten for PointCloud2 and Imu, matching the intended driver contract.

The cloud callback records header time, frame, `width * height` point count, and field names. The IMU callback records header time and frame. A periodic timer builds two `DiagnosticStatus` entries named `unitree_l1/cloud` and `unitree_l1/imu`.

A stream is initially a warning, becomes an error after the startup timeout with no messages, and becomes an error when its last arrival is too old. Low measured frequency is a warning. Non-increasing timestamps are errors. Zero stamps or excessive header age are warnings. Healthy streams report `stream healthy`.

Diagnostic key/value fields include received count, frequency, arrival age, header age, frame, timestamp counters, and cloud point metadata. The node publishes both statuses in one `DiagnosticArray` on `/diagnostics` and writes a concise periodic log line.

`test/test_stats.py`

The tests verify ten-hertz calculation from deterministic nanosecond arrivals, arrival and header ages, retention of point count, fields and frame, detection of zero and repeated stamps, and rejection of a window with fewer than two entries.

The ROS subscription and diagnostic integration are exercised by `scripts/monitor-synthetic-test.sh`, while the pure arithmetic remains in this fast package test.

`l1_octomap_bringup` **package**

This package owns the bridge between a Unitree-shaped PointCloud2 stream and upstream `octomap_server`. It owns the remapping, frames, parameter policy, stationary warning, combined launch composition, saved-map launch, and RViz profiles.

Complete package inventory

Path	Ownership and purpose	Installed or consumed by
ros2_ws/src/l1_octomap_bringup/package.xml	Project-owned OctoMap bringup dependency contract	rosdep, colcon, ament
ros2_ws/src/l1_octomap_bringup/setup.py	Project-owned installer for the package, launches, YAML, and RViz profiles	setuptools and colcon
ros2_ws/src/l1_octomap_bringup/setup.cfg	Project-owned script install paths	setuptools
ros2_ws/src/l1_octomap_bringup/resource/l1_octomap_bringup	Project-owned ament package marker	ament index
ros2_ws/src/l1_octomap_bringup/l1_octomap_bringup/__init__.py	Project-owned Python package marker	Python and setuptools
ros2_ws/src/l1_octomap_bringup/config/octomap.yaml	Project-owned occupancy server parameters	Live and saved-map launches
ros2_ws/src/l1_octomap_bringup/config/l1_octomap.rviz	Project-owned live cloud and occupied-voxel profile	Mapping RViz2
ros2_ws/src/l1_octomap_bringup/config/saved_octomap.rviz	Project-owned occupied-voxel-only profile	Saved-map RViz2
ros2_ws/src/l1_octomap_bringup/launch/l1_octomap.launch.py	Project-owned OctoMap launch for an existing cloud stream	octomap-launch.sh, combined launch
ros2_ws/src/l1_octomap_bringup/launch/unitree_l1_octomap.launch.py	Project-owned combined driver, monitor, map, and RViz graph	Advanced direct container launch
ros2_ws/src/l1_octomap_bringup/launch/view_saved_octomap.launch.py	Project-owned saved map server and optional viewer	view-octomap.sh
ros2_ws/src/l1_octomap_bringup/test/test_launch_file.py	Project-owned launch, scope, profile, and Docker checks	pytest through colcon
ros2_ws/src/l1_octomap_bringup/test/test_package_metadata.py	Project-owned runtime dependency and decoupling check	pytest through colcon
ros2_ws/src/l1_octomap_bringup/test/test_flake8.py	Project-owned Python style check	pytest through colcon
ros2_ws/src/l1_octomap_bringup/test/test_pep257.py	Project-owned docstring style check	pytest through colcon

config/octomap.yaml

The server's fixed frame is `map`; its sensor base frame is `unilidar_lidar`. Resolution is 0.10 metres and maximum sensor range is 15 metres. Occupancy update probabilities are hit 0.7, miss 0.4, minimum clamp 0.12, and maximum clamp 0.97.

Ground-plane filtering and speckle filtering are disabled, map compression is enabled, incremental 2D projection and free-space publication are disabled, and output is latched. Launch arguments can override frame, base frame, resolution, and maximum range without editing the YAML.

These values describe project policy, not immutable OctoMap behavior. A tuning change belongs here or in a documented launch override and should be validated against representative data.

`config/l1_octomap.rviz`

The live mapping profile uses fixed frame `map`. It shows the L1 PointCloud2 on `/unilidar/cloud` with Best Effort and Volatile QoS, plus occupied voxel markers on `/occupied_cells_vis_array` with Reliable and Transient Local QoS. Transient Local durability lets a newly opened RViz receive the server's most recent marker state. The profile also includes grid and map axes. It must receive a valid transform from `map` to the point-cloud frame before it can place the cloud.

`config/saved_octomap.rviz`

The saved-map profile uses fixed frame `map` and displays the occupied marker array without a live point cloud. It is intentionally simpler because a saved map does not require a sensor stream or live TF chain.

`launch/l1_octomap.launch.py`

The standalone mapping launch requires Docker and accepts `cloud_topic`, `world_frame`, `lidar_frame`, `resolution`, `max_range`, `static_sensor`, and `rviz`.

It starts upstream `octomap_server_node`, applies `octomap.yaml`, overlays selected public values, and remaps the server input `cloud_in` to the configured L1 cloud topic.

When `static_sensor=true`, it starts `tf2_ros/static_transform_publisher` as `l1_static_lidar_transform` with a zero translation and rotation from the world frame to the LiDAR frame. A launch message labels this identity transform as `bench-only`. When `static_sensor=false`, no transform is invented; a message states that an external time-varying transform is required.

The optional RViz node uses the live OctoMap profile. It is normally invoked inside the existing named runtime by `scripts/octomap-launch.sh`.

SAFETY / WARNING

The identity transform is invalid when the sensor or robot moves. OctoMap consumes pose; it does not estimate pose.

`launch/unitree_l1_octomap.launch.py`

The combined launch includes `l1_bringup` and `l1_octomap.launch.py` as two scoped groups. It passes common cloud and frame arguments, forces the child RViz arguments `false`, and creates one parent RViz using the mapping profile.

The scoping matters because both child launches declare an argument named `rviz`. Without scoped groups, a child's forced `false` value could overwrite the combined launch's public value before the parent RViz condition is evaluated.

The combined launch exposes port, cloud topic, world and LiDAR frames, resolution, range, stationary mode, and RViz. It always enables the monitor in its L1 child. It is installed for advanced container-side composition, while host-facing operational scripts use the separate long-running driver and map commands.

`launch/view_saved_octomap.launch.py`

The saved-map launch requires Docker, accepts an absolute container map path, defaults frame ID to `map`, and optionally opens the saved profile.

It starts upstream `octomap_server_node` with `octomap_path`, uses the same project server parameters, and assigns both frame and base frame to the saved-map frame. No hardware, live cloud, or pose transform is required.

`test/test_launch_file.py`

This test module parses the launch sources, checks every standalone mapping argument, verifies server selection and cloud remapping, requires TF and RViz components, and protects the bench warning and Docker guard.

It checks that the combined launch includes both project layers, keeps both child groups scoped, forces child RViz values `false`, and preserves a true parent RViz value in a launch-service test with stubbed children.

It also checks that the live RViz profile contains both cloud and occupied markers in frame `map`, and that the saved profile contains occupied markers but no PointCloud2 display.

`test/test_package_metadata.py`

This test parses `package.xml`, requires launch, launch_ros, l1_bringup, octomap_server, RViz2, and TF dependencies, and asserts that `unitree_lidar_ros2` is not directly coupled to the mapping package.

Host-facing scripts

The twenty scripts in `scripts/` are project-owned executable interfaces. They calculate repository paths from their own location, use strict shell options where appropriate, validate inputs, and call Docker or ROS only after checking relevant preconditions.

Build, shell, and boundary scripts

Path	Purpose	Main inputs	Main consumer or result
<code>scripts/assert-ros-container.sh</code>	Enforce supported ROS runtime	<code>REQUIRE_GUI</code>	Pass/fail verdict for all runtime wrappers
<code>scripts/docker-build.sh</code>	Build project image	Host UID/GID	<code>unitree-l1:humble-v1.0.16</code>
<code>scripts/docker-shell.sh</code>	Open supported interactive shell	GUI and LiDAR flags, optional device	Interactive <code>/workspace/ros2_ws</code> shell
<code>scripts/fetch-dependencies.sh</code>	Import and verify upstream clones	<code>dependencies.repos</code> and image	Clean pinned source trees
<code>scripts/workspace-build.sh</code>	Resolve and build six packages	Clean source tree and dependencies	Canonical build/install/log trees
<code>scripts/smoke-test.sh</code>	Check installed non-GUI runtime	Built workspace	<code>SMOKE_TEST_PASS</code>
<code>scripts/gui-smoke-test.sh</code>	Check X11, DRI, and RViz	Display, cookie, DRI devices	<code>GUI_SMOKE_TEST_PASS</code>
<code>scripts/verify-docker-only.sh</code>	Prove runtime process location	Runtime name, RViz requirement	Docker-only verdicts
<code>scripts/monitor-synthetic-test.sh</code>	Validate monitor without hardware	Optional isolated domain	<code>MONITOR_SYNTHETIC_HEALTH_PASS</code>

`scripts/assert-ros-container.sh`

This is the common runtime gate. It rejects a non-container environment, requires readable OS metadata, requires Ubuntu 22.04, sources Humble safely around unset variables, and requires ROS distribution `humble`.

It verifies that `ros2` and `rviz2` resolve beneath `/opt/ros/humble`. In GUI mode it also requires non-empty display variables, a readable X11 cookie, and `/dev/dri/renderD128`. Distinct exit codes make failures diagnosable.

`scripts/docker-build.sh`

The build wrapper exports the invoking user's UID and GID and runs Compose build with `--pull`. It consumes the strict Docker context and the digest-pinned recipe. It does not fetch source dependencies or compile the ROS workspace.

`scripts/docker-shell.sh`

The interactive shell defaults to GUI enabled and automatic LiDAR discovery. Flags can independently select GUI or headless mode and require, disable, or automatically attach hardware.

GUI mode validates the display, cookie, and both DRI character devices, then calculates the actual video and render GIDs. LiDAR mode prefers stable `/dev/serial/by-id` links, requires a unique candidate when requested, resolves the target, restricts accepted names to USB/ACM tty devices, and refuses a busy port without stopping its owner.

After composing the selected overlays, it runs the container assertion and opens Bash. Build-only work should use `--no-gui --no-lidar` to avoid irrelevant host dependencies.

`scripts/fetch-dependencies.sh`

The dependency wrapper runs `vcstool` inside the project image. It accepts a destination only when it is absent or an actual Git checkout. It imports missing repositories with `--skip-existing`.

Its verification function compares each nested HEAD with a hard-coded expected commit matching `dependencies.repos`, then rejects any nested `git status --porcelain` output. This protects against accidental vendor edits and partially reconstructed trees.

`scripts/workspace-build.sh`

Before any container build, the wrapper recursively searches `ros2_ws/src` for directories named `build`, `install`, or `log` and refuses to continue if it finds one. It fetches dependencies, repeats the check, and then runs `rosdep` and `colcon` in the base container.

Rosdep receives only the intended project and upstream ROS 2 roots. It ignores source-satisfied dependencies and skips the two already-verified keys `ament_python` and `pcl`. Colcon receives the same explicit base paths and requests symlink installation and cohesive console output.

The final in-container assertion again searches all source descendants. Thus an accidental output cannot be accepted merely because `.gitignore` hides it.

`scripts/smoke-test.sh`

The headless smoke test verifies the Docker runtime, distribution and OS, `ros2`, `colcon`, RViz executable availability, installed Unitree and monitor executables, bringup launch argument parsing, driver binary linkage, and PointCloud2 interface visibility. A pass means the installed software is coherent; it does not claim sensor data exists.

`scripts/gui-smoke-test.sh`

The GUI test calculates DRI group IDs, requires display authorization, composes the base and GUI services, validates the container, runs `xdpyinfo`, reports `glxinfo -B`, and checks RViz help. It verifies the rendering route without needing the L1.

`scripts/verify-docker-only.sh`

This script inspects a named running container, prints the host's limited role, verifies the container image and PID, and runs the common runtime assertion through `docker exec`.

When RViz is required, it uses `docker top` to find the process and inspects any host-visible `rviz2` PID to ensure its root filesystem contains `/.dockerenv`. Expected output includes `HOST_NATIVE_RVIZ_ABSENT` and `DOCKER_ONLY_PIPELINE_PASS`.

`scripts/monitor-synthetic-test.sh`

The test starts an isolated Compose run on domain 187 by default with localhost-only DDS. It launches the monitor, publishes a ten-hertz four-point cloud and a thirty-hertz IMU stream, receives one diagnostic message, and requires two healthy statuses plus correct point count and `x`, `y`, `z` fields.

Isolation prevents unrelated live nodes on the project's normal domain from contaminating the verdict. Temporary logs live inside the disposable container.

Hardware scripts

Path	Purpose	Important controls	Output
<code>scripts/check-lidar.sh</code>	Discover and explain USB serial candidates	Read-only udev, stat, and process inspection	Human-readable device evidence
<code>scripts/lidar-launch.sh</code>	Create the named live runtime	Unique safe tty, busy-port refusal, least privilege	Driver, monitor, optional raw RViz
<code>scripts/lidar-validate.sh</code>	Prove real stream health	Messages, types, rates, diagnostics	Timestamped ignored log and pass verdict

`scripts/check-lidar.sh`

The discovery script lists USB devices, waits for udev, gathers stable by-id links and `ttyUSB/ttyACM` candidates, resolves links, reports mode and numeric ownership, prints selected udev identity properties, and reports processes that already have the port open.

It changes no permissions, services, udev rules, network settings, or processes. Operators use its before-and-after output to identify the adapter rather than assuming `/dev/ttyUSB0`.

`scripts/lidar-launch.sh`

The live launcher defaults to container name `unitree_l1_runtime`, monitor enabled, and RViz disabled. It validates boolean inputs and the container name.

When `LIDAR_DEVICE` is unset, it requires exactly one stable by-id candidate. It resolves and validates the character device, refuses unexpected tty classes, reports and refuses an existing process owner, and refuses to replace an existing container with the chosen name.

It selects the LiDAR overlay and, when `START_RVIZ=true`, also selects the GUI overlay after checking the display and DRI devices. Before the long-running launch it starts an ephemeral container that proves the mapped device is readable and writable without privileged mode.

The final named container runs `l1_bringup` with the stable container port and selected monitor/RViz settings. `exec` preserves signal handling so `Ctrl-C` can shut down the launch cleanly.

`scripts/lidar-validate.sh`

The validator requires the named runtime to be active and creates `logs/tests/` on demand. Inside the runtime it lists nodes and typed topics, reads the driver port parameter, inspects cloud and IMU endpoints, and requires one message of the exact expected type from each stream.

It measures both topic rates for a bounded interval and requires `average rate:` output. It then requires one diagnostic array and prints `LIDAR_DATA_VALIDATION_PASS`. Host output is copied through `tee` to a timestamped ignored log.

Bag scripts

Path	Purpose	Safety boundary	Result
scripts/record-bag.sh	Record live cloud and IMU plus available context	Safe label, whole-second duration, running named container	New ignored bag directory
scripts/bag-info.sh	Inspect bag metadata	Existing bag strictly beneath project bags/	ros2 bag info report
scripts/replay-bag.sh	Replay with clock, monitor, and optional RViz	Existing finalized bag beneath bags/	Disposable replay graph

scripts/record-bag.sh

The recorder validates a safe label and non-negative whole-second duration, requires the named runtime, creates `bags/`, and forms a timestamped output directory beneath `/workspace/bags`.

Before recording, it requires one cloud and one IMU message. Those topics are always recorded. `/diagnostics`, `/tf`, and `/tf_static` are added only when present. An unbounded run stops with operator `Ctrl-C`; a bounded run uses `SIGINT` and a kill-after grace period so `rosbag2` can finalize `metadata.yaml`.

scripts/bag-info.sh

The inspector accepts exactly one path, resolves it, requires it to remain beneath the repository's `bags/` directory, requires `metadata.yaml`, translates the host path to `/workspace/bags/...`, and runs `ros2 bag info` in a headless disposable container.

The containment check prevents an arbitrary host path from being treated as project data.

scripts/replay-bag.sh

Replay applies the same path containment and metadata checks. It optionally adds the GUI overlay, starts `l1_monitor` with simulated time, optionally starts raw-cloud RViz with simulated time, and then plays the bag with `/clock`.

A cleanup trap terminates and waits for monitor and RViz child processes. No physical sensor or serial overlay is required.

OctoMap scripts

Path	Purpose	Main boundary	Result
scripts/octomap-launch.sh	Start live mapping in existing runtime	Stationary flag, duplicate-server check, GUI check	octomap_server and optional map RViz
scripts/evaluate-octomap.sh	Check non-empty live mapping output	Running server and transient-local messages	Mapping health verdict
scripts/save-octomap.sh	Save live tree safely	Safe extension, no overwrite, non-empty result	Ignored <code>.bt</code> or <code>.ot</code>
scripts/inspect-octomap.sh	Validate a saved file locally	Safe basename and valid header	Metadata and SHA-256
scripts/view-octomap.sh	Reopen map in isolated graph	Safe map, unique container/domain, optional GUI	Saved-map server and RViz

scripts/octomap-launch.sh

This wrapper operates on the already-running `unitree_l1_runtime`. It validates container name, `STATIC_SENSOR`, and `OCTOMAP_RVIZ`, checks package installation, and refuses to start a second `/octomap_server`.

`--check` stops after validating readiness. A normal run uses `docker exec` and launches the standalone mapping file with the chosen stationary and RViz settings.

When `OCTOMAP_RVIZ=true`, the common assertion requires GUI resources inside the existing container. Those resources can only have been attached when the runtime was created with the GUI Compose overlay. In the supported workflow, that means the runtime must have been started with `START_RVIZ=true`.

SAFETY / WARNING

Starting `START_RVIZ=true` and then `OCTOMAP_RVIZ=true` produces two intentional profiles: the first raw-cloud RViz and a second map-frame cloud-and-voxel RViz.

`scripts/evaluate-octomap.sh`

The evaluator requires the named runtime and `/octomap_server`. It classifies the graph as stationary bench mapping, mobile mapping requiring external pose, or saved-map replay based on the available nodes.

An embedded rcipy program subscribes with Reliable, Transient Local QoS to `/occupied_cells_vis_array` and `/octomap_binary`. It requires both messages, at least one marker containing points, and a non-empty binary map payload. The verdict reports marker counts, occupied points, resolution, payload size, tree ID, and frames.

The final message explicitly states that the check validates non-empty occupancy output, not SLAM trajectory accuracy.

`scripts/save-octomap.sh`

The saver accepts a safe `.bt` or `.ot` basename, creates `maps/`, refuses an existing file, requires the live runtime, and verifies the binary-map interface.

It invokes the official upstream `octomap_saver_node` with an absolute container path. After the call returns, the wrapper requires a non-empty host file and reports its byte count.

`scripts/inspect-octomap.sh`

The local inspector requires a readable safe-named file beneath `maps/`. It reads the text header before the binary payload and validates the OctoMap signature, tree ID, positive stored-node count, numeric resolution, and final `data` marker.

It reports format, tree ID, stored nodes, resolution, file size, modification time, and SHA-256. This confirms file structure and identity without starting ROS.

`scripts/view-octomap.sh`

The viewer first calls the inspector, then validates a separate viewer container name, an isolated ROS domain from zero to 232, and the RViz flag. `--check` validates inputs without starting a container.

GUI mode adds the GUI overlay after checking X11 and DRI. The map is exposed through the repository bind mount, and the container launches `view_saved_octomap.launch.py`. The default domain 43 keeps the saved map separate from the live domain 42.

Reconstructed upstream source

The two dependency directories are physically beneath `ros2_ws/src` so `colcon` can consume them, but they are not outer-repository source. Each directory is its own Git checkout with its own history and provenance.

Unitree UniLiDAR SDK

Property	Value
Destination	<code>ros2_ws/src/unilidar_sdk/</code>
Origin	<code>https://github.com/unitreerobotics/unilidar_sdk.git</code>
Commit	<code>1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6</code>
Release context	UniLiDAR SDK v1.0.16
Upstream-tracked files at pin	64
Colcon-selected subtree	<code>unitree_lidar_ros2/src/unitree_lidar_ros2</code>
Project modification policy	No local changes

The checkout contains upstream README and licence files, L1 images and frame documentation, a ROS 1 driver, a ROS 2 driver, and the raw Unitree LiDAR SDK. The raw SDK includes C++ examples, UDP examples, MAVLink headers, parser and SDK headers, prebuilt static libraries for `x86_64` and `aarch64`, and Windows artifacts.

The ROS 2 subtree contains its CMake definition, manifest, README, driver header and node source, vendor launch file, vendor RViz profile, and documentation image. It links the upstream static SDK into `unitree_lidar_ros2_node`.

The driver declares the serial port, yaw and range corrections, range limits, point-cloud scan count, topics, and frame names. It initializes the SDK at 2,000,000 baud, then publishes `PointCloud2` and `Imu` messages.

The project does not use the vendor launch file as its supported entry point because that launch hard-codes `/dev/ttyUSB0` and lacks the project monitor, container guard, and stable device mapping. `l1_bringup` supplies those adaptations without changing the checkout.

Only the ROS 2 driver subtree is listed in `colcon_defaults.yaml`. The ROS 1 package and raw CMake SDK must not be discovered as independent workspace packages.

An upstream tree may contain artifacts that resemble caches. Nested Git, not the outer ignore policy, determines whether an upstream-tracked file is part of the immutable checkout. Do not delete such a file selectively; restore or reconstruct the entire pin.

OctoMap mapping

Property	Value
Destination	ros2_ws/src/octomap_mapping/
Origin	https://github.com/OctoMap/octomap_mapping.git
Commit	f79da9a9a1fcdf82e72dab4df288d6cc27c6e163
Release context	octomap_mapping 2.3.1
Upstream-tracked files at pin	26
Colcon-selected subtree	Repository root
Project modification policy	No local changes

The checkout contains CI workflows, repository metadata, release notes, a build-dependency manifest, the `octomap_mapping` metapackage, and the `octomap_server` implementation.

`octomap_server` contains public headers and C++ sources for the standard, multilayer, static, and tracking server variants; the saver; launch XML; default parameters; package metadata; API documentation; and an eraser command-line utility.

Colcon receives the repository root and discovers two packages: the metapackage `octomap_mapping` and executable package `octomap_server`. Project launch files use `octomap_server_node` and the official saver rather than copying their source.

Project-specific cloud remapping, frame conventions, resolution, sensor range, stationary warning, and RViz profile remain in `ll_octomap_bringup`. A required upstream bug fix should be proposed upstream or pinned as a documented fork; it should not appear as an unexplained dirty nested checkout.

Container-supplied dependencies

The source tree alone is not the complete execution environment. The Dockerfile installs ROS and native libraries whose files reside in the image.

Layer	Examples	Source of truth
Base OS and ROS	Ubuntu 22.04, ROS 2 Humble	Digest-pinned ROS base image
Build tools	GCC/G++, CMake, Ninja, pkg-config, colcon	Dockerfile apt list
Dependency tools	rosdep, vcstool, Git	Dockerfile apt list
Point-cloud stack	PCL, Boost, ROS PCL conversion packages	Dockerfile apt list
Mapping stack	OctoMap library, <code>octomap_msgs</code> , <code>octomap_ros</code>	Dockerfile apt list
Runtime tools	<code>launch_ros</code> , <code>tf2_ros</code> , <code>rosbag2</code> , <code>ros2doctor</code>	Dockerfile apt list
Visualization	RViz2, Mesa DRI, X11 and xauth utilities	Dockerfile apt list
Hardware inspection	<code>usbutils</code> , <code>iproute2</code> , <code>ping</code>	Dockerfile apt list

Package manifests declare ROS-level dependencies. The Dockerfile ensures the supported image actually contains the development and runtime packages required by the pinned source. Both layers must be reviewed when a new dependency is added.

Generated and ignored paths

Generated paths are expected during normal work and must remain outside source directories. Only the three reviewed manual PDFs are retained in Git.

Canonical generated directories

Path	Producer	Typical contents	Reconstruction or lifecycle
<code>ros2_ws/build/</code>	colcon, CMake, setuptools, tests	Per-package intermediates, binaries, test results	Rebuild with <code>workspace-build.sh</code> and <code>colcon test</code>
<code>ros2_ws/install/</code>	colcon	Installed package shares, executables, environment hooks	Rebuild with <code>workspace-build.sh</code>
<code>ros2_ws/log/</code>	colcon	Command and event logs	Recreated by colcon
<code>bags/</code>	<code>record-bag.sh</code>	roslaunch metadata and storage files	Record again from live topics
<code>maps/</code>	<code>save-octomap.sh</code>	Binary <code>.bt</code> or full <code>.ot</code> occupancy trees	Save again from a running map
<code>logs/</code>	<code>lidar-validate.sh</code> and local checks	Timestamped validation output	Regenerate by rerunning checks
<code>exports/manuals/</code>	Manual renderer	Three tracked, reviewed PDFs	Regenerate and review from maintained source
Other <code>exports/paths</code>	Operator	Optional generated exports	Ignored; regenerate as required

`ros2_ws/build/`, `ros2_ws/install/`, and `ros2_ws/log/` may exist after a valid build. Their presence at the workspace root is normal. The same names beneath `ros2_ws/src` are always invalid.

Ignored file and cache classes

Pattern or path	Meaning	Policy
compile_commands.json	Generated compiler database	Recreate from build tooling
__pycache__/, *.pyc, *.pyo, *.pyd	Python bytecode	Never treat as authored source
*.egg-info/	Setuptools package metadata	Recreate during build
.coverage, htmlcov/	Coverage output	Local test evidence only
.mypy_cache/, .pytest_cache/, .ruff_cache/	Tool caches	Delete or recreate freely outside nested pins
.env, .env.*, *.xauth	Local settings or authorization	Never commit credentials
.idea/, .vscode/	IDE state	Local only
.DS_Store, Thumbs.db, *~	OS and editor artifacts	Local only
*.log, *.zip	Generated logs and archives	Keep outside project history

Why generated outputs are not source

Build output embeds compiler versions, absolute paths, timestamps, platform details, and intermediate state. Install output duplicates source and generated binaries. Logs and runtime artifacts may expose local device identifiers or become very large. Bags and maps represent a particular environment rather than the reusable implementation.

Keeping these products out of Git makes a change review describe the actual change. Reproducibility comes from the Dockerfile, manifests, immutable pins, package source, and build commands, not from committing one machine's build tree.

ENGINEERING NOTE

A generated artifact may be valuable evidence without becoming maintained source. Preserve it in an appropriate external archive when required.

Build-time interactions

The supported build is a chain of small, auditable controls.

```
.dockerignore
|
v
docker/Dockerfile + docker/entrypoint.sh
|
v
unitree-l1:humble-v1.0.16 image
|
+-- docker/compose.yaml bind-mounts repository at /workspace
|
v
scripts/fetch-dependencies.sh
|
+-- config/dependencies.repos
+-- ros2_ws/src/unilidar_sdk/
+-- ros2_ws/src/octomap_mapping/
|
v
scripts/workspace-build.sh
|
+-- rosdep over five permitted roots
+-- ros2_ws/colcon_defaults.yaml
+-- three project packages
+-- three reconstructed packages
|
+-- ros2_ws/build/
+-- ros2_ws/install/
+-- ros2_ws/log/
```

The outer repository supplies recipes and project code. The nested repositories supply immutable upstream source. The image supplies the compiler, ROS distribution, and system dependencies. Colcon combines them into generated workspace outputs.

The entrypoint later sources `install/setup.bash`, making the six packages visible to run-time commands. Removing the install tree does not remove source; it only requires a rebuild.

Run-time component interactions

Live sensor data flow

```

Physical Unitree L1
|
v
Unitree adapter and host ttyUSB/ttyACM device
|
v
docker/compose.lidar.yaml
|
v
/dev/unitree_lidar inside unitree_l1_runtime
|
v
unitree_lidar_ros2_node
|-- /unilidar/cloud  sensor_msgs/msg/PointCloud2
|   |
|   |-- ll_monitor
|   |-- raw-cloud RViz profile
|   |-- octomap_server cloud_in remapping
|   |-- rosbag2 recorder
|
|-- /unilidar/imu  sensor_msgs/msg/Imu
|   |
|   |-- ll_monitor
|   |-- rosbag2 recorder

```

The driver is the only component that talks to the serial device. The monitor and mapping server consume ROS messages and need no device access of their own.

Diagnostic flow

```

/unilidar/cloud + /unilidar/imu
|
v
ll_monitor
|
|-- arrival and header timing
|-- frequency thresholds
|-- point count and field names
|-- frame and timestamp checks
|
v
/diagnostics  diagnostic_msgs/msg/DiagnosticArray
|
|-- lidar-validate.sh
|-- record-bag.sh when topic exists

```

Diagnostics summarize transport and metadata health. They do not recalibrate, repair, or republish the source messages.

Occupancy mapping flow

```

/unilidar/cloud
|
|-- PointCloud2 frame_id: unilidar_lidar
|
map -> unilidar_lidar transform
|
|-- static identity only for stationary bench mode
|-- dynamic external pose for moving mode
|
v
octomap_server
|
|-- /occupied_cells_vis_array
|-- /octomap_binary message and service interface
|-- live map-frame RViz profile
|-- evaluate-octomap.sh
|-- octomap_saver_node
|
v
maps/name.bt or maps/name.ot

```

OctoMap integrates sensor rays in a fixed world frame. A missing or incorrect transform prevents meaningful accumulation. An identity transform is correct only while the sensor is perfectly stationary relative to map.

RViz profiles and GUI ownership

Profile	Source file	Fixed frame	Displays	How it starts
Raw L1	ll_bringup/ config/ unitree_ ll.rviz	unilidar_lidar	Grid, axes, live cloud	START_RVIZ=true ./scripts/lidar-launch.sh
Live OctoMap	ll_octomap_ bringup/ config/ll_ octomap.rviz	map	Grid, axes, live cloud, occupied voxels	OCTOMAP_RVIZ=true ./scripts/octomap-launch.sh
Saved OctoMap	ll_octomap_ bringup/ config/ saved_ octomap.rviz	map	Grid, axes, saved occupied voxels	./scripts/view-octomap.sh map.bt

The live OctoMap wrapper uses `docker exec` inside the named hardware runtime. Because device and mount sets are fixed at container creation, its RViz option requires that runtime to have been launched with GUI support. The current supported launcher couples GUI support to `START_RVIZ=true`, so the raw and mapping RViz windows coexist during that workflow.

The saved-map viewer creates its own container with the GUI overlay and therefore does not depend on the hardware runtime.

Bag and map lifecycle

Artifact	Creation	Validation	Consumption	Removal
Bag directory	record-bag.sh	bag-info.sh, positive topic counts	replay-bag.sh	Operator-controlled; ignored by Git
Live OctoMap	octomap-launch.sh	evaluate-octomap.sh	RViz and saver	Ends with mapping process
.bt or .ot map	save-octomap.sh	inspect-octomap.sh	view-octomap.sh	Operator-controlled; ignored by Git
Validation log	lidar-validate.sh	Pass markers and review	Local evidence	Operator-controlled; ignored by Git
Manual PDF	build-manuals.sh	Renderer checks	Readers	Regenerate from Markdown

Where future changes belong

The correct directory follows ownership and lifecycle, not convenience.

Placement decision table

Change	Placement rule
Driver launch argument or project default	Use ll_bringup/launch/ or ll_bringup/config/, never unilidar_sdk/
Stream-health calculation	Use ll_monitor/ll_monitor/, not a driver callback or shell script
Diagnostic threshold	Use ll_bringup/config/unitree_ll.yaml with rationale, not hard-coded vendor source
OctoMap parameter	Use ll_octomap_bringup/config/octomap.yaml, not upstream octomap_server/params/
Cloud remapping or frame composition	Use ll_octomap_bringup/launch/, not vendor launch files
Raw or mapping visualization choice	Use the corresponding project .rviz file, not user-local RViz state
Host preflight or supported workflow	Use scripts/, not ad hoc commands preserved only in generated logs
Image system dependency	Use docker/Dockerfile, not host installation instructions
GUI or device access	Use a narrow Compose overlay, not the base service or privileged mode
New external repository pin	Use config/dependencies.repos and version docs, not a copied source tree
Architecture rationale	Use docs/decisions.md, not code comments alone
Hardware safety or connection step	Use docs/hardware-runbook.md, not a README-only note
Reproducible check and status	Use docs/validation-matrix.md, not an untracked log only
New package test	Use that package's test/ directory, not repository root
Bag, map, log, or report output	Use an ignored runtime/export directory, never ros2_ws/src/
Reviewed project manual PDF	Use one of the three tracked names in exports/manuals/

Adding a project-owned ROS 2 package

A new reusable ROS node or integration layer belongs under `ros2_ws/src/<project_package>/`. It needs a manifest, supported build definition, tests, licence metadata, and appropriate ament resource registration.

The package must be added deliberately to both `ros2_ws/colcon_defaults.yaml` and the explicit roots in `scripts/workspace-build.sh`. Its dependencies must exist in the image or a pinned source repository. Smoke or integration coverage should prove the installed package rather than only importing it from source.

Do not broaden discovery to all of `ros2_ws/src` merely to avoid listing the package. The explicit list is what prevents ROS 1 and raw SDK components in the Unitree clone from entering the build.

Updating a pinned dependency

An upgrade should change the vcstool commit, matching verification constants in `fetch-dependencies.sh`, release context in `versions-lock.md`, and any compatibility or source references affected by the update.

The clean nested checkout must build with all six packages, pass project tests and smoke tests, and receive hardware validation when its behavior touches live communication. Any necessary adaptation belongs in a project layer unless a documented fork is intentionally adopted.

Adding a launch or configuration file

Place a launch file in the owning package's `launch/` directory and parameters or RViz state in `config/`. Update that package's `setup.py` data files so the artifact reaches the install share. Add a test that verifies its public arguments or critical settings.

Host-facing users should normally invoke a validating script. Direct launch commands are useful inside the supported container, but they must preserve the Docker guard and ownership boundary.

Adding a host script

A supported operational script belongs in `scripts/`, should be executable, should derive the repository root from `BASH_SOURCE`, and should validate user input before changing state. It should use exact paths, safe basenames, explicit container names, and least-privilege Compose overlays.

Scripts should not stop unrelated processes, modify global permissions, disable host services, or silently overwrite runtime artifacts. A command that only diagnoses should remain read-only.

Adding documentation or publication material

Concise maintained Markdown belongs under `docs/`. Manual source belongs under `docs/manuals/`. The three reviewed PDFs belong under `exports/manuals/` and are tracked; other rendered exports remain ignored.

Do not commit copied upstream manuals, large evidence archives, raw bags, maps, or generated reports as substitutes for sources and reconstruction instructions.

Reconstruction from a clean clone

The project is complete when the tracked source, pinned nested source, container image, and generated workspace can be recreated without relying on old build artifacts.

1. Select the audited project source

```
git clone https://github.com/pokepascu/Unitree-L1-ROS2-Octomap.git
cd Unitree-L1-ROS2-Octomap
git checkout 07616c2
```

At this point the three project packages, Docker recipe, scripts, lock file, and documentation exist. The two upstream directories and all generated outputs may correctly be absent.

2. Build the supported image

```
./scripts/docker-build.sh
```

This consumes only the Docker recipe and entrypoint through the restricted build context. It produces the supported Jammy/Humble toolchain without installing ROS on the host.

3. Reconstruct upstream source

```
./scripts/fetch-dependencies.sh
```

Expected output identifies both clean commits. The resulting directories are ignored by outer Git but are independent nested Git repositories.

```
git -C ros2_ws/src/unilidar_sdk rev-parse HEAD
git -C ros2_ws/src/octomap_mapping rev-parse HEAD
git -C ros2_ws/src/unilidar_sdk status --short
git -C ros2_ws/src/octomap_mapping status --short
```

The two HEAD values must match the pins, and the two status commands must print nothing.

4. Build the workspace

```
./scripts/workspace-build.sh
```

This creates only:

```
ros2_ws/build/
ros2_ws/install/
ros2_ws/log/
```

It must not create:

```
ros2_ws/src/build/
ros2_ws/src/install/
ros2_ws/src/log/
```

5. Run data-independent checks

```
./scripts/smoke-test.sh
./scripts/monitor-synthetic-test.sh
```

When X11 and DRI are available:

```
./scripts/gui-smoke-test.sh
```

Project package tests can be run in a headless shell:

```
./scripts/docker-shell.sh --no-gui --no-lidar
cd /workspace/ros2_ws
colcon test --packages-select ll_bringup ll_monitor ll_octomap_bringup
colcon test-result --all --verbose
```

6. Recreate runtime artifacts only when needed

Bags require live messages or a deliberate test graph. Live maps require a point cloud and valid TF. Saved-map viewing requires an existing `.bt` or `.ot`. Validation logs require rerunning their check. PDFs require maintained manual source and the renderer.

None of these products is required to prove that the source tree is complete.

Repository hygiene

Verify the audited manifest

After later publication commits add files, use the audited tree object rather than current working-tree count:

```
git ls-tree -r --name-only 07616c2 | wc -l
git ls-tree -r --name-only 07616c2
```

The count for the audited commit is 73.

Verify current outer source status

```
git status --short
git diff --check
```

The first command should show only intentional current work. The second should print nothing.

Detect forbidden source outputs

```
find ros2_ws/src -type d \
  \( -name build -o -name install -o -name log \) -print
```

This command must print nothing. It searches inside project packages and reconstructed dependencies because the placement rule applies to the entire source tree.

Detect generated files tracked by outer Git

```
git ls-files | grep -E \
  '(^|/)(build|install|log|__pycache__|\.pytest_cache)(/|$)|\.pyc$'
```

This command must print nothing for outer-project source. Nested dependency content is governed by its own Git index and immutable pin.

Verify dependency integrity

```
test "$(git -C ros2_ws/src/unilidar_sdk rev-parse HEAD)" = \
  1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6

test "$(git -C ros2_ws/src/octomap_mapping rev-parse HEAD)" = \
  f79da9a9a1fcdf82e72dab4df288d6cc27c6e163

test -z "$(git -C ros2_ws/src/unilidar_sdk status --porcelain)"
test -z "$(git -C ros2_ws/src/octomap_mapping status --porcelain)"
```

The supported wrapper performs equivalent checks before a workspace build.

Verify package discovery

Inside a project container:

```
cd /workspace/ros2_ws
colcon list
```

Expected inventory:

```
l1_bringup
l1_monitor
l1_octomap_bringup
octomap_mapping
octomap_server
unitree_lidar_ros2
```

Unexpected ROS 1, raw SDK, generated, or duplicate packages indicate that the discovery boundary has been broadened incorrectly.

Verify ignore intent

```
git check-ignore -v \
  ros2_ws/src/unilidar_sdk \
  ros2_ws/src/octomap_mapping \
  ros2_ws/build \
  ros2_ws/install \
  ros2_ws/log \
  bags maps logs exports
```

Existing or representative paths should resolve to the intended `.gitignore` rule. Remember that an ignore match does not validate physical placement beneath `src`.

Complete manifest reconciliation

This final manifest groups the 73 tracked files by responsibility and provides a count check.

Group and count	Files
Root policy and entry, 4	<code>.dockerignore</code> , <code>.gitignore</code> , <code>LICENSE</code> , <code>README.md</code>
Dependency lock, 1	<code>config/dependencies.repos</code>
Container, 5	<code>docker/Dockerfile</code> , three Compose YAML files, <code>docker/entrypoint.sh</code>
Technical docs, 6	Six Markdown files in <code>docs/</code>
Workspace control, 1	<code>ros2_ws/colcon_defaults.yaml</code>
<code>l1_bringup</code> , 11	Manifest, setup files, marker, module marker, launch, two configs, three tests
<code>l1_monitor</code> , 10	Manifest, setup files, marker, module marker, two modules, three tests
<code>l1_octomap_bringup</code> , 15	Manifest, setup files, marker, module marker, three launches, three configs, four tests
Host scripts, 20	Twenty executable shell scripts
Total, 73	Audited project source at 07616c2

Root, configuration, Docker, and documentation paths

```
.dockerignore
.gitignore
LICENSE
README.md
config/dependencies.repos
docker/Dockerfile
docker/compose.gui.yaml
docker/compose.lidar.yaml
docker/compose.yaml
docker/entrypoint.sh
docs/decisions.md
docs/hardware-runbook.md
docs/project-structure.md
docs/sources.md
docs/validation-matrix.md
docs/versions-lock.md
```

Workspace and project-package paths

```
ros2_ws/colcon_defaults.yaml
ros2_ws/src/l1_bringup/config/unitree_l1.rviz
ros2_ws/src/l1_bringup/config/unitree_l1.yaml
ros2_ws/src/l1_bringup/l1_bringup/__init__.py
ros2_ws/src/l1_bringup/launch/unitree_l1.launch.py
ros2_ws/src/l1_bringup/package.xml
ros2_ws/src/l1_bringup/resource/l1_bringup
ros2_ws/src/l1_bringup/setup.cfg
ros2_ws/src/l1_bringup/setup.py
ros2_ws/src/l1_bringup/test/test_flake8.py
```

```
ros2_ws/src/l1_bringup/test/test_launch_file.py
ros2_ws/src/l1_bringup/test/test_pep257.py
ros2_ws/src/l1_monitor/l1_monitor/__init__.py
ros2_ws/src/l1_monitor/l1_monitor/monitor_node.py
ros2_ws/src/l1_monitor/l1_monitor/stats.py
ros2_ws/src/l1_monitor/package.xml
ros2_ws/src/l1_monitor/resource/l1_monitor
ros2_ws/src/l1_monitor/setup.cfg
ros2_ws/src/l1_monitor/setup.py
ros2_ws/src/l1_monitor/test/test_flake8.py
ros2_ws/src/l1_monitor/test/test_pep257.py
ros2_ws/src/l1_monitor/test/test_stats.py
ros2_ws/src/l1_octomap_bringup/config/l1_octomap.rviz
ros2_ws/src/l1_octomap_bringup/config/octomap.yaml
ros2_ws/src/l1_octomap_bringup/config/saved_octomap.rviz
ros2_ws/src/l1_octomap_bringup/l1_octomap_bringup/__init__.py
ros2_ws/src/l1_octomap_bringup/launch/l1_octomap.launch.py
ros2_ws/src/l1_octomap_bringup/launch/unitree_l1_octomap.launch.py
ros2_ws/src/l1_octomap_bringup/launch/view_saved_octomap.launch.py
ros2_ws/src/l1_octomap_bringup/package.xml
ros2_ws/src/l1_octomap_bringup/resource/l1_octomap_bringup
ros2_ws/src/l1_octomap_bringup/setup.cfg
ros2_ws/src/l1_octomap_bringup/setup.py
ros2_ws/src/l1_octomap_bringup/test/test_flake8.py
ros2_ws/src/l1_octomap_bringup/test/test_launch_file.py
ros2_ws/src/l1_octomap_bringup/test/test_package_metadata.py
ros2_ws/src/l1_octomap_bringup/test/test_pep257.py
```

Executable script paths

```
scripts/assert-ros-container.sh
scripts/bag-info.sh
scripts/check-lidar.sh
scripts/docker-build.sh
scripts/docker-shell.sh
scripts/evaluate-octomap.sh
scripts/fetch-dependencies.sh
scripts/gui-smoke-test.sh
scripts/inspect-octomap.sh
scripts/lidar-launch.sh
scripts/lidar-validate.sh
scripts/monitor-synthetic-test.sh
scripts/octomap-launch.sh
scripts/record-bag.sh
scripts/replay-bag.sh
scripts/save-octomap.sh
scripts/smoke-test.sh
scripts/verify-docker-only.sh
scripts/view-octomap.sh
scripts/workspace-build.sh
```

EXPECTED RESULT

Every maintained file at commit 07616c2 has an assigned owner, purpose, consumer, and reconstruction or lifecycle rule.